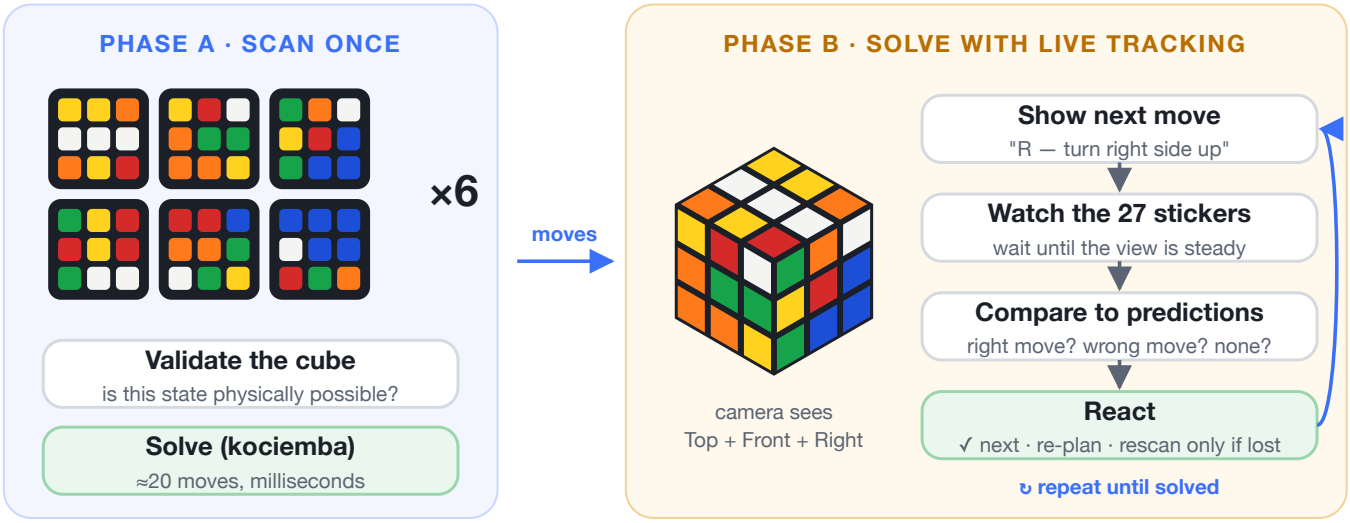
RUBIK'S CUBE SOLVER · PLAN

Live Camera Solver Pipeline

You scan the cube once. The app works out the solution, then watches you through the camera while you solve. When you do the right move it moves on. If you do a wrong move, it figures out **which** move you did, re-plans, and tells you what to do next.



1 · Scan once

Show each of the 6 sides one time, at the start. After that, no more scanning unless tracking gets lost.

2 · Solve

The cube is checked and solved in about 20 moves — in milliseconds.

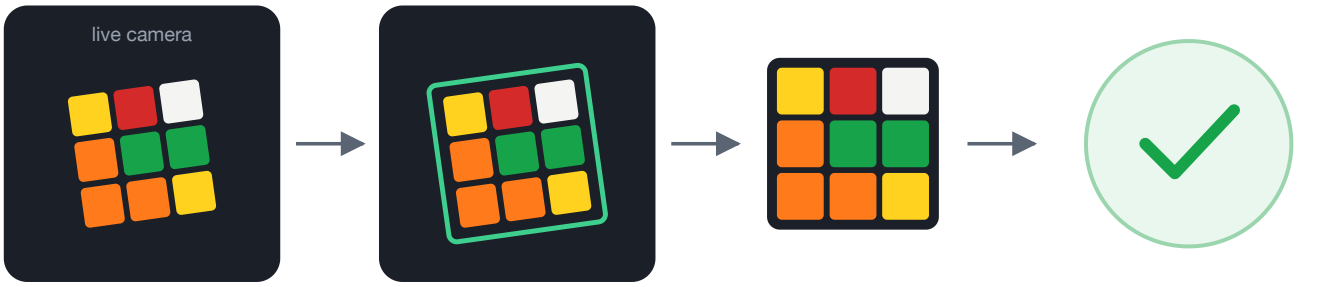
3 · Track live

Hold the cube tilted. Every move you make is checked; mistakes trigger a re-plan.

PHASE A

Scan the cube once

This full scan happens **once, at the start**. Like today, but with no Capture button and no grid to line up — the camera finds the side by itself.



1 · Camera frame | 2 · Find 9 stickers | 3 · Straighten + read | 4 · Steady ½ s → captured

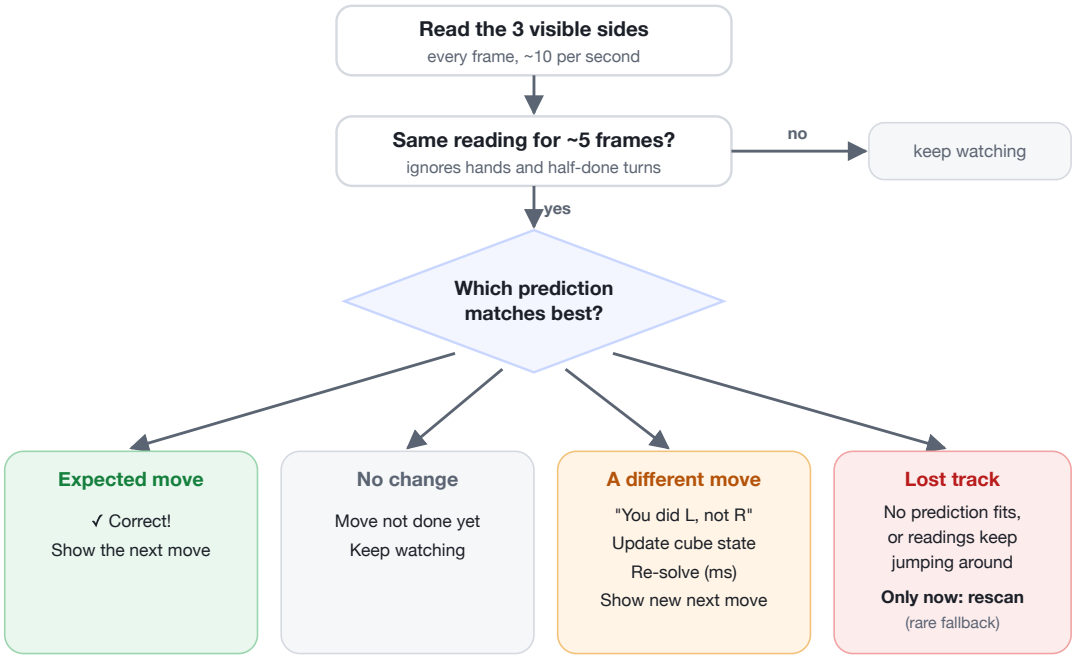
1. **Camera on.** Your browser streams the computer camera to the Python server about 10 times per second.
2. **Find the side.** In each frame the server looks for 9 small squares arranged in a 3×3 grid.
3. **Straighten and read.** The side is "flattened" (even if tilted) and each sticker's color is read by comparing it to the cube's own center stickers.
4. **Auto-capture.** When the same 9 colors are seen for about half a second, the side is captured and shown in the bottom panel.
5. **Repeat ×6.** A side shown twice is rejected (same center color), exactly as it works now.
6. **Check + solve.** The app confirms the cube is physically possible, then computes the solution (~20 moves).

> After the scan you hold the cube in one **home position** for the rest of the solve: same side up, same side facing you, tilted so the right side shows (see page 4).

PHASE B

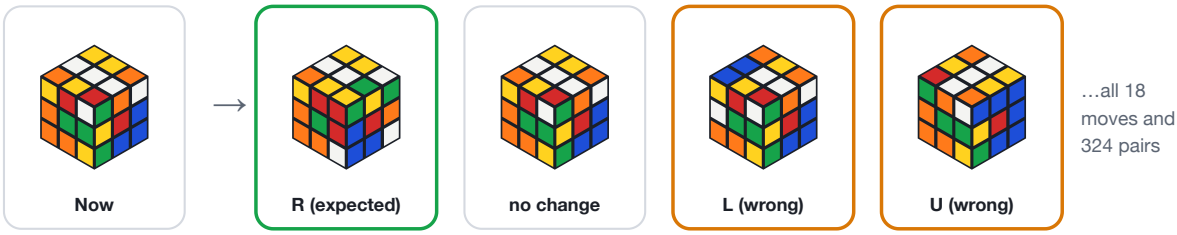
Solve with live tracking

The app always knows the full cube state, so it can predict what the camera **should** see after every possible move — then check which one actually happened.



The trick: compare against predictions

From the current state, the app builds what the 3 visible sides would look like after each possibility, then picks the closest to what the camera sees.



When does it ask for a rescan?

Almost never. The first scan is the only one you normally do. A rescan is only requested when the tracker can no longer trust what it sees:

🔍 No prediction fits

What the camera sees doesn't match the correct move, no move, or any 1–2 wrong moves (e.g. the whole cube was rotated).

〰 Readings jump around

The best match keeps flipping between options for a few seconds — a sign the colors are being misread.

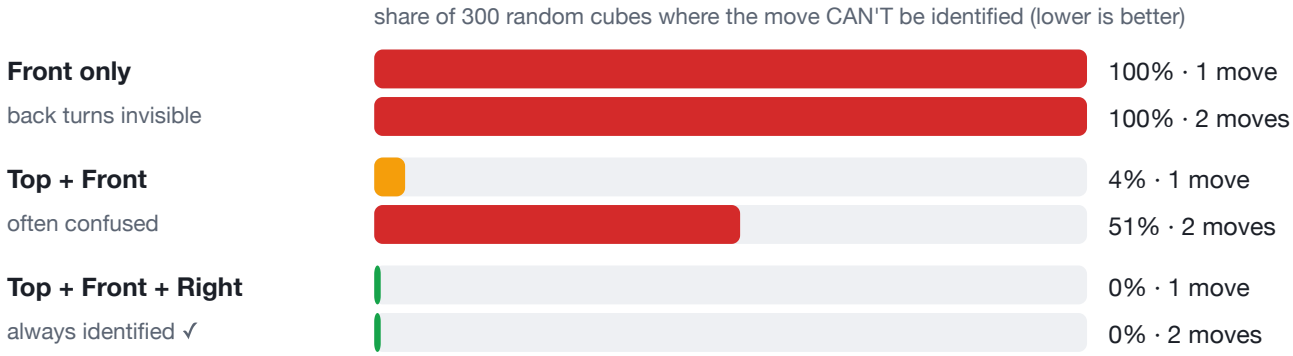
👀 Cube out of view

Put the cube down? Hold it back in the home position. If it still matches, tracking simply continues — no rescan.

WHY IT WORKS

Why the cube must show 3 sides

With one camera you only see part of the cube. We tested 300 random cubes with your app's own cube model: can the app tell exactly which move you did?



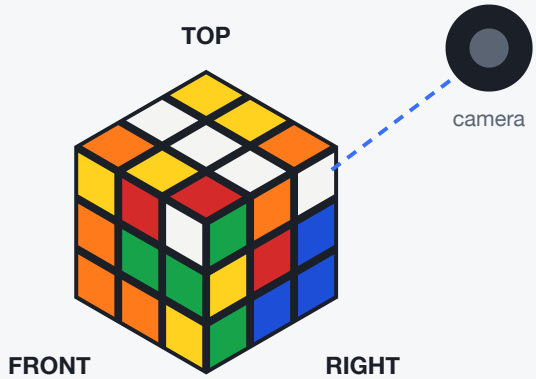
What this means

Front only: a turn of the back side changes nothing the camera can see — impossible to track.

Top + Front: works most of the time, but two quick moves are confused about half the time.

Top + Front + Right: every one of the 18 possible moves — and every pair of moves — looks different. **Always identified.**

> So during solving, hold the cube corner-on to the camera like the picture, and only turn layers — don't rotate the whole cube.



Home position: the camera looks at the corner where Top, Front and Right meet.

Rules for you while solving

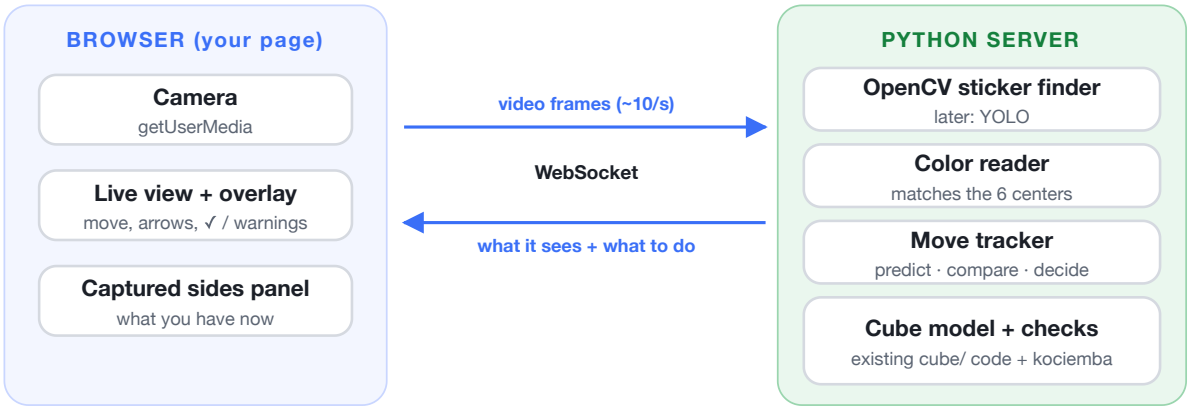
Same side up and facing you for the whole solve, tilted so the right side shows.

Turn layers only. Rotating the whole cube makes it lose track and ask for a rescan.

Even light. Strong direct light causes glare that hides colors.

WHAT WE'LL USE

Tools, and why each one



PART	TOOL	WHY
Camera + screen	**Your current web page**	Already works; nothing new to install for you.
Sending frames	**WebSocket (FastAPI)**	A continuous fast stream instead of one request per photo.
Finding stickers	**OpenCV (Python)**	Standard vision library: finds square shapes, straightens tilted sides.
Reading colors	**Your center-based matcher**	Already handles lighting; tracking only compares to predictions.
Cube model + checks	**Your existing cube/ code**	Already correct and tested — reused as is.
Solving	**kociemba**	~20 moves vs 60–150 today; re-plans in milliseconds.
Finding the cube (later)	**YOLO (Ultralytics)**	Learns the cube's corners even with fingers on it.

Build order — the app keeps working after every step

1. **Swap in the kociemba solver** — Small
   ~20-move solutions; fast enough to re-plan instantly.
2. **Live video + auto-capture scan** — Medium
   Phase A. No Capture button, no grid to line up.
3. **Read 3 sides from a tilted cube** — Hardest
   Find the 27 stickers in the corner-on view.
4. **Move tracking + re-plan + rescan fallback** — Medium
   Phase B: compare to predictions, re-solve after mistakes.
5. **YOLO cube detector (only if needed)** — Optional
   If OpenCV struggles with hands covering the cube.

Note: kociemba is GPL-licensed — fine for personal use; it matters only if the app is ever sold or released as closed source.